

Rapport d'analyse technique

Organisation du travail

Le projet a été réalisé seul. Le travail a été organisé en suivant l'ordre des quêtes pour avancer de manière logique. La première étape a été de comprendre le sujet et de vérifier les fichiers fournis. Comme seule la table des véhicules était disponible, il a fallu reconstruire entièrement la base de données en créant les tables client, station et location. Une fois la base prête, les requêtes ont été écrites et testées dans pgAdmin jusqu'à obtenir des résultats corrects.

GitHub a servi à sauvegarder régulièrement le projet, garder un historique clair et structurer les fichiers.

L'environnement de travail reposait sur pgAdmin pour exécuter les requêtes et sur VS Code pour écrire les scripts SQL et le README.

Analyse du modèle de données

La base de données repose sur quatre tables : vehicule, client, station et location.

Chaque table a un rôle précis dans le fonctionnement du service de location.

La table vehicule contient les informations sur les véhicules électriques.

La table client regroupe les utilisateurs du service.

La table station représente les points de départ et d'arrivée.

La table location enregistre chaque location et relie les trois autres tables.

Les relations sont simples : un client peut faire plusieurs locations, un véhicule peut être loué plusieurs fois, et chaque location possède une station de départ et une station d'arrivée. Les clés primaires et étrangères permettent de relier correctement les données. Cette structure permet de gérer la flotte, de suivre les locations et d'analyser l'activité du service.

Choix techniques et syntaxe SQL

Les requêtes simples ont été réalisées avec SELECT, WHERE, ORDER BY et LIMIT.

Pour les requêtes plus avancées, la méthode a été de partir d'une version simple puis d'ajouter les jointures ou les filtres nécessaires.

Les jointures ont été utilisées pour relier les tables entre elles.

LEFT JOIN a servi à trouver les véhicules jamais loués.

GROUP BY a permis de regrouper les données, par exemple par ville ou par client.

HAVING a été utilisé pour filtrer après un regroupement, notamment pour les clients ayant fait plusieurs locations.

Les filtres ont été construits en traduisant les besoins du sujet en conditions SQL. Par exemple, pour afficher les véhicules disponibles, il fallait filtrer sur l'état "Disponible", et pour afficher les véhicules les plus autonomes, il fallait trier par autonomie décroissante.

Difficultés rencontrées et solutions

La base complète n'était pas fournie, ce qui a été la première difficulté. Il a donc fallu créer les tables manquantes et ajouter des données fictives pour pouvoir réaliser les quêtes.

Après l'import du CSV, la table vehicule ne possédait pas de clé primaire, ce qui empêchait de créer la table location. La solution a été de supprimer les doublons puis d'ajouter une clé primaire sur id_vehicule.

Certaines requêtes ne renvoyaient aucun résultat à cause de différences entre majuscules et minuscules dans la colonne etat. Une vérification des valeurs distinctes a permis de corriger les filtres.

Les requêtes utilisant GROUP BY et HAVING ont demandé un peu plus de réflexion. Elles ont été résolues en testant chaque partie de la requête séparément pour comprendre le fonctionnement du regroupement.

Conclusion

Le projet a permis de comprendre la structure d'une base relationnelle, d'écrire des requêtes SQL simples et intermédiaires, de reconstruire une base complète à partir d'un fichier partiel et d'utiliser GitHub pour organiser le travail. L'ensemble du projet a servi d'introduction solide à la manipulation de bases de données.